Manual Implementation Guide: Microsoft Entra ID, Azure RBAC, and Azure SQL

Document date: 20 October 2026

Table of Contents

- 1. Purpose
- 2. Required Implementation Order
- 3. Scope and Current Deployment Assumptions
- 4. Required Separation of Duties
- 5. Identity Inventory and Naming
- 6. Create the Protected API App Registration
- 7. Create Only the Selected SPA App Registrations
- 8. Create and Assign Security Groups
- 9. Configure the Bootstrap Administrator
- 10. Create the Runtime Managed Identities
- 11. Configure Azure SQL Entra Authentication and Permissions
- 12. Configure AWReason API Access
- 13. Optional GitHub Actions Setup
- 14. Environment File Handoff
- 15. Acceptance Checklist
- 16. Security Notes
- 17. Repository References
- 18. Microsoft References

1. Purpose

This document defines the identity and access configuration required to deploy and operate TalentMatch in Azure. It is intended for customer teams that will manually create or approve:

- Microsoft Entra ID app registrations and enterprise applications, including the optional delegated Microsoft 365 source connector.
- Entra security groups and application-role assignments.
- The optional GitHub Actions deployment identity and federated credentials, only when GitHub Actions will deploy the application.
- User-assigned managed identities for the selected application stacks.
- Azure role-based access control (RBAC) assignments.
- The Azure SQL Microsoft Entra administrator and database permissions.
- The non-secret identifiers that must be returned to the deployment team for the target `.env` file and, only when used, GitHub environment configuration.

This guide does not require application code changes.

2. Required Implementation Order

Use this order to avoid circular dependencies and partial authorization:

1. Confirm deployment method (CLI or GitHub Actions), tenant, subscription, environment, `STACK_TARGET`, authentication and reuse flags, names, URLs, and responsible operators. Agree the inventory in section 3.1 before creating objects.
2. Create the protected API registration, delegated scope, and four app roles.
3. Create or reuse only the selected SPA registration(s), add their redirect URIs, request the API delegated scope, and grant admin consent.
4. Create the application security groups and assign them to protected API roles.
5. Select the bootstrap administrator and configure it as the SQL Entra administrator.
6. Create or reuse only the selected runtime user-assigned managed identities, or authorize Terraform to create those identities.
7. Grant the actual operator/deployment identity the Azure and Entra rights required for the selected operations. **Only for GitHub Actions**, create or reuse its deployment app and configure GitHub OIDC federated credentials.
8. Populate the `.env` profile. **Only for GitHub Actions**, also populate the applicable GitHub OIDC secrets and the complete `DEPLOYMENT_ENV_FILE` secret.
9. Run a reviewed shared Terraform plan.
10. Apply shared infrastructure so the App Services and identity attachments exist.
11. Only with `AWR_AUTH_MODE=entra`, assign each selected runtime managed identity to the AWReason API application role, either through Terraform or through the agreed manual-and-import process.
12. From the approved VNet-connected host, create the selected Azure SQL contained user(s) and grant each the three database roles.
13. Run the bootstrap administrator check/apply/check workflow.
14. Deploy only the selected application package(s), using the same environment file throughout.
15. Test sign-in, token claims, database connectivity, AWReason token acquisition, and access revocation.
16. Remove temporary privileged access and retain the approved handoff record.

3. Scope and Current Deployment Assumptions

The repository supports either application stack independently or both together:

Stack	Technology	Runtime identity
Stack A	React and Node.js/Express	One user-assigned managed identity
Stack B	Blazor WebAssembly and ASP.NET Core	One user-assigned managed identity

Each selected stack:

- Authenticates workforce users from one Entra tenant.
- Uses its own public SPA app registration.
- Requests a delegated scope from one shared protected API registration.
- Validates the same four application roles.
- Connects to the shared Azure SQL database with managed identity.
- Can call the AWReason API with managed identity when `AWR_AUTH_MODE=entra`.

3.1 Select the Deployment Inventory First

Set `STACK_TARGET` in the applicable environment file (`.env_local`, `.env_qa`, or `.env_prod`) before creating any identities. This guide assumes `APP_AUTH_MODE=entra`.

GitHub Actions is optional. Deploying through the repository's CLI scripts does not require a GitHub deployment app registration, GitHub OIDC federated credentials, or GitHub environment secrets/variables. The chosen CLI operator or deployment identity still needs the Azure and Entra permissions for the operations it performs. See section 13 for the deployment-method distinction.

Required object or grant	`STACK_TARGET=stack-a`	`STACK_TARGET=stack-b`	`STACK_TARGET=both`
Protected API registration and enterprise application	One shared pair	One shared pair	One shared pair
Stack A SPA registration, enterprise application, delegated permission and consent	Required	Do not create	Required
Stack B SPA registration, enterprise application, delegated permission and consent	Do not create	Required	Required
Runtime user-assigned managed identity	Stack A only	Stack B only	One per stack
SQL contained user and three database-role memberships	Stack A identity only	Stack B identity only	Both identities
AWReason application-role assignment, only with `AWR_AUTH_MODE=entra`	One, for Stack A	One, for Stack B	Two, one per identity
Protected API scope, four app roles, user/group assignments and bootstrap administrator	One shared set	One shared set	One shared set
Optional GitHub OIDC deployment identity	Only for GitHub Actions: one created or reused identity per environment, not per stack	Same optional requirement	Same optional requirement

The product registration total is **two for a single stack** (API plus its SPA) and **three for both stacks**. These totals exclude the separately managed GitHub deployment identity, the runtime managed identities, and the optional Microsoft 365 connector. Do not duplicate the protected API or user-role groups per stack.

`STACK_TARGET` controls shared Terraform provisioning, not just which application package is deployed. The deployment command's target may select a subset of the enabled stacks; it cannot enable a stack excluded by the environment file. For example, a Stack B-only profile must use `STACK_TARGET=stack-b` even when running a `shared-only` command to prepare its shared dependencies. `STACK_TARGET=shared-only` is a different profile: it provisions shared resources and, with Entra authentication, the protected API, but neither SPA nor runtime identity.

Reuse flags are independent:

- `ENTRA_REUSE=TRUE`: reuse the shared API and only the selected SPA registration(s); do not create product registrations.
- `AZ_IDENTITIES_REUSE=TRUE`: reuse only the selected runtime identity or identities; do not create runtime identities. This does not enable app registration reuse.
- `ENTRA_CREATE_ROLE_GROUPS=FALSE`: do not create baseline role groups.
- `AWR_AUTH_MODE=apikey` or `none`: do not create managed-identity AWReason app-role assignments, even if an old `AWR_API_CLIENT_ID` remains populated.
- `MICROSOFT365_SOURCE_MODE=off` or `mock`: do not create a real Microsoft 365 connector or its federated credentials. Select `graph` only when that optional feature is required.

`APP_AUTH_MODE=simple` skips product Entra registrations, role groups and the bootstrap API app-role assignment. It does not disable SQL managed identities or separately enabled AWReason/connector authentication. Simple authentication is for supported development/testing use, not a production alternative to this guide's Entra configuration.

Existing-environment warning: changing `STACK_TARGET`, authentication mode, or create/reuse ownership can plan deletion of previously Terraform-managed objects. Reused objects are data sources, not deletion targets. Review the shared plan and coordinate state migration before changing an existing both-stack environment to a single stack; do not apply an unexpected deletion.

3.2 Environment Templates and Manual Handoff

The checked-in `.env_qa.example` file is a deployment template, not evidence of the currently deployed QA configuration. Its current defaults indicate:

- `APP_AUTH_MODE=entra`
- `AWR_AUTH_MODE=apikey`
- `STACK_TARGET=both`.
- Azure SQL, the selected application managed identities, and the selected product Entra registrations are configured to be created rather than reused.
- Automatic SQL bootstrap is enabled.
- An existing VNet, integration subnet, and SQL private endpoint are reused.
- Key Vault and API Management are disabled for this environment.

The manual customer handoff profile in this guide is intentionally different: it reuses customer-created registrations, identities, and Azure SQL, and disables automatic SQL bootstrap. When that profile is selected, the customer SQL team must execute the database grants in section 11 from a host that can reach the private SQL endpoint.

The repository now also contains infrastructure for a separate, optional multi-tenant Microsoft 365 source connector. It is not one of the product sign-in/API registrations. Its Stack B infrastructure and automated gate are implemented, but its live gate and B-GATE remain pending. Do not treat it as production-approved until those gates pass. Follow Delegated Microsoft 365 bulk-scoring connector registration when `MICROSOFT365_SOURCE_MODE=graph`. That mode also requires an enabled or reused Key Vault wrapping key; the checked-in QA defaults do not enable it.

4. Required Separation of Duties

Use separate operators where practical. Activate privileged roles through Privileged Identity Management for the implementation window and remove temporary access after verification.

Activity	Minimum practical permission
Create or update the selected product app registrations and, only if enabled, Microsoft 365 connector	Owner of the specific applications, Cloud Application Administrator, or Application Administrator
Create or update connector federated identity credentials	Owner of the connector application, Cloud Application Administrator, Application Administrator, Global Administrator, Hybrid Identity Administrator, or an approved custom role containing `microsoft.directory/applications/credentials/update`
Create app roles and expose the API scope	Owner of the protected API application, Cloud Application Administrator, or Application Administrator
Grant delegated admin consent to each selected SPA	Cloud Application Administrator or Application Administrator, subject to tenant consent policy
Assign users or groups to protected API app roles	Owner of the protected API service principal, Cloud Application Administrator, Application Administrator, or User Administrator
Create and manage the application security groups	Owner of the specific groups or Groups Administrator
Create user-assigned managed identities	Managed Identity Contributor on the identity resource group, or Contributor on that resource group
Assign an existing managed identity to App Service	Managed Identity Operator on the identity plus write access to the App Service
Set the Azure SQL Entra administrator	Azure role containing `Microsoft.Sql/servers/administrators/read` and `Microsoft.Sql/servers/administrators/write` at the SQL server scope

Activity	Minimum practical permission
Create SQL contained users and grant database roles	The configured SQL Entra administrator, or a temporary database principal with equivalent user, schema, and role-management rights
Deploy infrastructure into an existing resource group	Contributor on the deployment resource group
Create the deployment resource group	Contributor at subscription scope, or a custom role that permits resource-group creation
Create or change subnets, private endpoints, and private DNS	Network Contributor on the VNet resource group
Read reused resources without modifying them	Reader on each external resource group
Deploy application ZIP packages only	Website Contributor on the selected target App Service(s)

Azure RBAC does not grant Entra directory permissions, and Entra directory roles do not grant Azure resource permissions. Both permission planes must be configured when the same deployment identity runs the AzureRM and AzureAD Terraform providers.

Ordinary TalentMatch users require no Azure subscription role, no Azure SQL login, and no Entra directory role.

5. Identity Inventory and Naming

Use names approved for the selected `PROJECT_NAME` and `ENVIRONMENT` in the untracked deployment profile. The table references inputs or naming patterns, not concrete deployment coordinates.

Copy destinations: unless stated otherwise, copy values to the named variables in the applicable deployment profile (`.env_local`, `.env_qa`, or `.env_prod`). **Handoff only** means there is no deployment environment variable for that value: retain it for audit, assignments or application administration. **Runtime** settings are populated by Terraform on App Service, not manually added to the deployment profile. GitHub secrets are separate destinations and apply only when GitHub Actions is used.

Object	Recommended display name	Applicability	Required output and copy destination
Protected API app registration	`<PROJECT_NAME>-<ENVIRONMENT>-api`	Any Entra-authenticated stack	Client ID -> `ENTRA_API_APP_CLIENT_ID`; identifier URI -> `ENTRA_API_IDENTIFIER_URI`; application object ID: handoff only
Protected API enterprise application	Same approved name as the API registration	Any Entra-authenticated stack	Service principal object ID -> `ENTRA_API_SERVICE_PRINCIPAL_OBJECT_ID`
Stack A SPA registration	`<PROJECT_NAME>-<ENVIRONMENT>-stack-a`	Stack A or both only	Client ID -> `ENTRA_STACK_A_CLIENT_ID`; redirects -> `ENTRA_STACK_A_REDIRECT_URIS`
Stack B SPA registration	`<PROJECT_NAME>-<ENVIRONMENT>-stack-b`	Stack B or both only	Client ID -> `ENTRA_STACK_B_CLIENT_ID`; redirects -> `ENTRA_STACK_B_REDIRECT_URIS`
Optional Microsoft 365 source connector	`<PROJECT_NAME>-<ENVIRONMENT>-microsoft365-source`	`MICROSOFT365_SOURCE_MODE=graph` only	Client ID -> `MICROSOFT365_SOURCE_CLIENT_ID`; callbacks -> `MICROSOFT365_SOURCE_REDIRECT_URIS`; application/service principal object IDs: handoff only
Stack A managed identity	`AZ_IDENTITY_STACK_A_NAME`	Stack A or both only	Name -> `AZ_IDENTITY_STACK_A_NAME`; resource group -> `AZ_IDENTITIES_RG`; resource/client/principal IDs: see section 10
Stack B managed identity	`AZ_IDENTITY_STACK_B_NAME`	Stack B or both only	Name -> `AZ_IDENTITY_STACK_B_NAME`; resource group -> `AZ_IDENTITIES_RG`; resource/client/principal IDs: see section 10

Object	Recommended display name	Applicability	Required output and copy destination
Optional GitHub deployment app registration	Customer-approved deployment identity name	Only for GitHub Actions; create or reuse once per environment	Client ID -> GitHub secret `AZURE_CLIENT_ID`; service principal object ID: handoff only
Global Admin group	Customer-approved global-admin group name	Shared user authorization, if group-based assignment is used	Group object ID: handoff/application mapping only; no `.env` variable
Organization Admin group	Customer-approved organization-admin group name	Shared user authorization, if group-based assignment is used	Group object ID: handoff/application mapping only; no `.env` variable
Recruiter group	Customer-approved organization/department recruiter group name	Shared user authorization, if group-based assignment is used	Group object ID: handoff/application mapping only; no `.env` variable
Business Panel group	Customer-approved organization business-panel group name	Shared user authorization, if group-based assignment is used	Group object ID: handoff/application mapping only; no `.env` variable

Names are presentation values. All handoffs and automated mappings must use immutable object IDs and application IDs.

Assign at least two durable customer-controlled owners to every app registration, enterprise application, and security group.

6. Create the Protected API App Registration

In the Microsoft Entra admin center:

1. Go to **Entra ID > App registrations > New registration**.
2. Name the registration `talentmatch-<environment>-api`.
3. Select **Accounts in this organizational directory only**.
4. Do not configure a redirect URI for the protected API.
5. Record:
   - **Directory (tenant) ID** -> `AZURE_TENANT_ID`.
   - **Application (client) ID** -> `ENTRA_API_APP_CLIENT_ID`.
   - **Object ID** from this app registration -> handoff only; do not copy it to `ENTRA_API_SERVICE_PRINCIPAL_OBJECT_ID`.
6. Open **Expose an API**.
7. Set an Application ID URI. The recommended value is:

   ```
   api://<protected-api-application-client-id>
   ```

   A customer-approved custom URI is also supported, but it must be used consistently. The applications construct the requested scope from the exact identifier URI and scope value. Copy the complete **Application ID URI** to `ENTRA_API_IDENTIFIER_URI`; do not append `/access_as_user` to that variable.
8. Add one delegated scope:

Field	Required value
Scope name	`access_as_user` -> `ENTRA_API_SCOPE`
Who can consent	Admins only
Admin consent display name	`Access TalentMatch as a user`
Admin consent description	`Allow the application to access TalentMatch as the signed-in user.`
State	Enabled

9. Record the delegated scope UUID from the approved API manifest as `ENTRA_API_SCOPE_ID` in the untracked deployment profile. Keep its scope value separately in `ENTRA_API_SCOPE`. Both creation and reuse require an explicit scope UUID; there is no repository default. For an existing registration, preserve its current UUID. For a new registration, select and record a UUID once before the first Terraform plan; never regenerate it on subsequent deployments.
10. Under **Manifest**, confirm `requestedAccessTokenVersion` is `2`.
11. Do not create a client secret or certificate for this API registration unless a separate, approved confidential-client scenario requires one.

6.1 Create the Four App Roles

Under **App roles**, create the following enabled roles. Set **Allowed member types** to **Users/Groups**.

Display name	Value required by the application	Description	Copy the actual role ID to
Admin	`admin`	Full platform administration across all organizations.	`ENTRA_ADMIN_APP_ROLE_ID`
Organization Admin	`organization_admin`	Delegated administration within assigned organizations.	`ENTRA_ORGANIZATION_ADMIN_APP_ROLE_ID`
Recruiter	`recruiter`	Recruiting operations within assigned organization and department scopes.	`ENTRA_RECRUITER_APP_ROLE_ID`
Business Panel	`business_panel`	Business panel review within assigned organization and department scopes.	`ENTRA_BUSINESS_PANEL_APP_ROLE_ID`

Record the actual IDs from the approved API manifest in the untracked profile. There are no repository defaults. Supply all four distinct role ID variables together for both creation and reuse. Copy the role **UUID**, not its display name or value such as `admin`. The role values must match the table exactly.

Do not delete and recreate roles after production use. The immutable role IDs are persisted deployment inputs.

6.2 Record the Enterprise Application

1. Go to **Entra ID > Enterprise applications > All applications**.
2. Open the enterprise application corresponding to the protected API.
3. Record its **Object ID**. This is the service principal object ID, not the application client ID and not the app registration object ID. Copy it to `ENTRA_API_SERVICE_PRINCIPAL_OBJECT_ID`.
4. Under **Properties**, keep **Assignment required?** set to **No** to match the current deployment definition.

TalentMatch denies access by default unless all required authorization checks pass. A valid token alone does not grant TalentMatch access; the user must also have an allowed app-role claim and an active application assignment in the shared database.

7. Create Only the Selected SPA App Registrations

Follow these steps only for the SPA(s) selected by `STACK_TARGET`:

- `stack-a`: create or reuse `talentmatch-<environment>-stack-a` only.
- `stack-b`: create or reuse `talentmatch-<environment>-stack-b` only.
- `both`: create or reuse both registrations.

Do not create the other stack's registration, enterprise application, redirect URIs, permission grant or admin consent as a prerequisite for a single-stack deployment. A runtime managed identity does not replace a browser SPA registration; they serve different authentication flows.

1. Go to **Entra ID > App registrations > New registration**.
2. Select **Accounts in this organizational directory only**.
3. Under **Authentication**, add the **Single-page application** platform.
4. Add the exact approved HTTPS redirect URI(s) from the selected stack's `ENTRA_STACK_A_REDIRECT_URIS` or `ENTRA_STACK_B_REDIRECT_URIS` in the untracked deployment profile. Do not infer a hostname from a project or environment name.
5. Add every approved custom-domain origin separately.
6. Use the exact scheme, host, port, path, and trailing-slash form supplied to the deployment team.
7. Do not add a Web platform.
8. Do not create a client secret or certificate. SPAs are public clients and use authorization code flow with PKCE.
9. Do not enable the implicit grant checkboxes for access tokens or ID tokens.
10. Under **API permissions**, select **Add a permission > My APIs**, select the protected API, choose **Delegated permissions**, and add `access_as_user`.
11. Grant admin consent for the tenant for the delegated permission.
12. Confirm the permission status is **Granted for <tenant>**.
13. Record each selected SPA's **Application (client) ID** and registered redirect URIs using this mapping:

Selected SPA	Copy Application (client) ID to	Copy exact SPA redirect URIs to
Stack A	`ENTRA_STACK_A_CLIENT_ID`	`ENTRA_STACK_A_REDIRECT_URIS`
Stack B	`ENTRA_STACK_B_CLIENT_ID`	`ENTRA_STACK_B_REDIRECT_URIS`

    For multiple redirect URIs, use a comma-separated list. Do not copy an app registration's Object ID or enterprise application Object ID into a SPA client ID variable. Leave the unselected stack's variables absent or empty.

No Microsoft Graph runtime permissions are required by the TalentMatch SPAs. The four API app roles and user/group assignments are shared, not recreated for each SPA.

8. Create and Assign Security Groups

Group-based assignment to enterprise applications requires Microsoft Entra ID P1 or P2. If the tenant does not have the required license, assign users directly to the protected API roles and retain the same application database authorization process.

8.1 Group Design

Create security-enabled groups dedicated to TalentMatch. Recommended patterns:

Access	Group pattern	API role
Global platform administrators	`talentmatch-<env>-admin`	`admin`
Administrators for one organization	`talentmatch-<env>-<org>-organization-admin`	`organization_admin`
Recruiters for one department	`talentmatch-<env>-<org>-<department>-recruiter`	`recruiter`
Business panel for one organization	`talentmatch-<env>-<org>-business-panel`	`business_panel`

Requirements:

- Use direct user membership.
- Do not rely on nested groups; nested membership does not cascade for enterprise-application assignment.
- Assign each group to exactly one TalentMatch app role.
- Create separate groups where organization or department scope differs.
- Do not add the managed identities to user-role groups.
- Record every group object ID, intended role, organization, and department.

Group object IDs have no deployment `.env` variable. Retain them in the handoff record and use them in TalentMatch's group-to-business-scope mappings; do not put a group ID into an app-role ID variable.

8.2 Assign Groups to the API Enterprise Application

For each group:

1. Open **Enterprise applications > talentmatch-<environment>-api**.
2. Select **Users and groups > Add user/group**.
3. Select the group.
4. Select exactly one of the four app roles.
5. Select **Assign**.
6. Verify the assignment under **Users and groups**.

The Entra assignment supplies the coarse `roles` token claim. It does not by itself authorize access to an organization or department. The application administrator must also map the group object ID to the correct business scope using the repository's role-management procedure.

After initial bootstrap, routine user and scope administration should be performed through TalentMatch rather than by granting Azure RBAC or SQL access.

9. Configure the Bootstrap Administrator

Select one enabled Entra workforce **Member** user. Do not use a guest account.

The current repository safety contract requires this same user to be:

1. The Azure SQL logical server's Microsoft Entra administrator.
2. Directly assigned the protected API `admin` app role.
3. Identified by `ENTRA_BOOTSTRAP_ADMIN_OBJECT_ID`.

Record and copy:

Value	Copy destination
Bootstrap user's user principal name (UPN)	`SQL_AAD_ADMIN_LOGIN`
Bootstrap user's Entra Object ID	Both `ENTRA_BOOTSTRAP_ADMIN_OBJECT_ID` and `SQL_AAD_ADMIN_OBJECT_ID`; the values must be identical
Approved initial organization name	`ENTRA_BOOTSTRAP_ORGANIZATION_NAME`
Approved initial department name	`ENTRA_BOOTSTRAP_DEPARTMENT_NAME`
Account enabled status and `userType=Member` confirmation	Handoff only; no `.env` variable

The organization and department variables take business names, not Entra group names or object IDs.

The deployment currently expects Terraform to manage the bootstrap user's app role assignment even when `ENTRA_REUSE=TRUE`. If the customer manually creates the same assignment before deployment, the deployment team must import that assignment into the shared Terraform state before applying. Do not create the same assignment independently without agreeing the Terraform-state ownership.

10. Create the Runtime Managed Identities

Create or reuse only the user-assigned managed identities selected in the environment file, in the target subscription and tenant:

`STACK_TARGET`	Identity required
`stack-a`	`id-talentmatch-node-<environment>` only
`stack-b`	`id-talentmatch-blazor-<environment>` only
`both`	Both identities above

For each selected identity, record and copy:

Value	Deployment profile destination	Other use
Stack A identity resource name	`AZ_IDENTITY_STACK_A_NAME`, only when reusing Stack A	Manual SQL helper: `STACK_A_IDENTITY_NAME`
Stack B identity resource name	`AZ_IDENTITY_STACK_B_NAME`, only when reusing Stack B	Manual SQL helper: `STACK_B_IDENTITY_NAME`
Identity resource group name	`AZ_IDENTITIES_RG`, when reusing selected identities	Both reused identities must be in this configured resource group
Azure resource ID	No direct `.env` input; Terraform resolves it from identity name and resource group	Handoff and App Service identity attachment
Application (client) ID	No direct `.env` input; Terraform resolves it	Terraform sets runtime `AZURE_CLIENT_ID` on that stack's App Service
Principal (object) ID	No direct `.env` input; Terraform resolves it	Handoff, Azure RBAC and AWReason app-role assignments

The `STACK_A_IDENTITY_NAME`/`STACK_B_IDENTITY_NAME` variables are shell inputs for the manual SQL helper, not deployment-profile settings. Use `AZ_IDENTITY_STACK_A_NAME`/`AZ_IDENTITY_STACK_B_NAME` in the deployment profile.

The client ID and principal object ID are different values:

- The client ID is used by the application credential selection.
- The principal object ID is used for Azure role assignments and Entra application-role assignments.
- The resource ID is used to attach the identity to an App Service.

If these identities are manually pre-created, set `AZ_IDENTITIES_REUSE=TRUE` and provide the shared identity resource group plus the identity name for each stack selected by `STACK_TARGET`. Stack A-only and Stack B-only deployments do not require the other stack's identity.

The infrastructure deployment attaches the identities to their respective App Services. A separately operated deployment identity needs Managed Identity Operator on these identities and sufficient write permission on the App Services.

11. Configure Azure SQL Entra Authentication and Permissions

11.1 Set the SQL Logical Server Entra Administrator

Copy the reused database coordinates and administrator values as follows:

Azure value	Copy destination
SQL logical server resource name, without `.database.windows.net`	`SQL_SERVER_NAME`
Application database name	`SQL_DATABASE_NAME`
SQL server's resource group name	`SQL_RG`
SQL Entra administrator's UPN	`SQL_AAD_ADMIN_LOGIN`
SQL Entra administrator's user Object ID	`SQL_AAD_ADMIN_OBJECT_ID`, identical to `ENTRA_BOOTSTRAP_ADMIN_OBJECT_ID`
SQL server's full DNS name, for example `<sql-server>.database.windows.net`	Manual SQL helper shell variable `SQL_SERVER_FQDN`; Terraform supplies runtime `AZURE_SQL_SERVER_FQDN`

Do not put the full DNS name into `SQL_SERVER_NAME`, or the identity resource group into `SQL_RG`. The manual helper uses `SQL_DATABASE_NAME` as well, while Terraform supplies runtime `AZURE_SQL_DATABASE_NAME` on each selected app.

On the reused Azure SQL logical server:

1. Open the SQL server in the Azure portal.
2. Select **Microsoft Entra ID** or **Microsoft Entra admin**.
3. Select the bootstrap administrator defined in section 9.
4. Save the configuration.
5. Verify the configured administrator object ID exactly matches `ENTRA_BOOTSTRAP_ADMIN_OBJECT_ID`.

The control-plane operator needs a role containing:

```
Microsoft.Sql/servers/administrators/read
Microsoft.Sql/servers/administrators/write
```

Scope this permission to the SQL logical server where possible.

11.2 Connect from an Approved Bootstrap Host

The active environment uses a private SQL endpoint and has `AZ_SQL_BOOTSTRAP_ENABLED=FALSE`. Run the following from a VNet-connected host that:

- Resolves the SQL private DNS name.
- Can connect to TCP 1433.
- Is signed in as the configured SQL Entra administrator.
- Can acquire a token for `https://database.windows.net/.default`.

Do not use the SQL administrator for application runtime connections.

11.3 Create the Managed Identity Database Users

Connect to the application database, not `master`. Create the shared schema once:

```
IF NOT EXISTS (SELECT 1 FROM sys.schemas WHERE name = N'talentmatch')
    EXEC(N'CREATE SCHEMA [talentmatch]');
```

Stack A or both only: run this block with the exact Stack A managed identity display name. Skip it entirely for Stack B-only deployments.

```
IF NOT EXISTS (
    SELECT 1 FROM sys.database_principals
    WHERE name = N'id-talentmatch-node-test'
)
    CREATE USER [id-talentmatch-node-test] FROM EXTERNAL PROVIDER;

ALTER ROLE [db_datareader] ADD MEMBER [id-talentmatch-node-test];
ALTER ROLE [db_datawriter] ADD MEMBER [id-talentmatch-node-test];
ALTER ROLE [db_ddladmin] ADD MEMBER [id-talentmatch-node-test];
```

Stack B or both only: run this block with the exact Stack B managed identity display name. Skip it entirely for Stack A-only deployments.

```
IF NOT EXISTS (
    SELECT 1 FROM sys.database_principals
    WHERE name = N'id-talentmatch-blazor-test'
)
    CREATE USER [id-talentmatch-blazor-test] FROM EXTERNAL PROVIDER;

ALTER ROLE [db_datareader] ADD MEMBER [id-talentmatch-blazor-test];
ALTER ROLE [db_datawriter] ADD MEMBER [id-talentmatch-blazor-test];
ALTER ROLE [db_ddladmin] ADD MEMBER [id-talentmatch-blazor-test];
```

For repeatable execution, use the repository helper. It performs membership checks before each `ALTER ROLE` and is preferred when Node.js dependencies are available:

```
export SQL_SERVER_FQDN="<sql-server>.database.windows.net"
export SQL_DATABASE_NAME="<application-database>"
```

Then run exactly one of the following, matching the environment file. The helper reads identity-name variables, not `STACK_TARGET`; clear the unused variable to prevent a stale shell value from granting access to another stack.

Stack A only:

```
unset STACK_B_IDENTITY_NAME
export STACK_A_IDENTITY_NAME="id-talentmatch-node-<environment>"
node infra/scripts/bootstrap-sql-entra-users.mjs
```

Stack B only:

```
unset STACK_A_IDENTITY_NAME
export STACK_B_IDENTITY_NAME="id-talentmatch-blazor-<environment>"
node infra/scripts/bootstrap-sql-entra-users.mjs
```

Both stacks:

```
export STACK_A_IDENTITY_NAME="id-talentmatch-node-<environment>"
export STACK_B_IDENTITY_NAME="id-talentmatch-blazor-<environment>"

node infra/scripts/bootstrap-sql-entra-users.mjs
```

The required application database roles are:

Role	Reason
`db_datareader`	Read application data
`db_datawriter`	Insert, update, and delete application data
`db_ddladmin`	Create and migrate the `talentmatch` schema at application startup

Do not grant either runtime managed identity:

- `db_owner`
- SQL server administrator
- Azure SQL Contributor
- Subscription or resource-group Contributor
- A SQL username or password

The temporary bootstrap operator can use the configured Entra administrator or `db_owner` on this database only. Remove temporary database grants after the managed identity users have been verified.

11.4 Verify SQL Permissions

Run:

```
SELECT
    member_principal.name AS principal_name,
    role_principal.name AS database_role
FROM sys.database_role_members AS drm
JOIN sys.database_principals AS role_principal
    ON role_principal.principal_id = drm.role_principal_id
JOIN sys.database_principals AS member_principal
```

```
    ON member_principal.principal_id = drm.member_principal_id
WHERE member_principal.name IN (
    N'id-talentmatch-node-test',
    N'id-talentmatch-blazor-test'
)
ORDER BY member_principal.name, role_principal.name;
```

Expected result: three rows for a single-stack deployment, or six for both stacks: `db_datareader`, `db_datawriter`, and `db_ddladmin` for each selected identity. An unselected stack does not need a contained user or any SQL grants.

If `CREATE USER ... FROM EXTERNAL PROVIDER` cannot resolve the managed identities, confirm:

- The SQL server and identities belong to the same tenant.
- The managed identity names are correct and unambiguous.
- The operator is the configured Entra administrator.
- The SQL server identity has the Microsoft Graph read permissions required by the customer's Azure SQL identity-resolution policy.

For automated service-principal execution of `CREATE USER`, Microsoft recommends a SQL server identity with the minimum Microsoft Graph permissions needed to resolve users, groups, and applications. Directory Readers is a broader fallback and should be approved only when the least-privilege Graph permissions cannot be used.

12. Configure AWReason API Access

This section applies only to environments that select `AWR_AUTH_MODE=entra`. The current checked-in QA and Production templates select `AWR_AUTH_MODE=apikey`.

The customer must provide:

AWReason value	Copy destination
API Application (client) ID	`AWR_API_CLIENT_ID`
API Application ID URI	`AWR_AAD_AUDIENCE`, with `/.default` appended, for example `api://<awreason-api-client-id>/.default`
Enterprise application Object ID	Handoff only; Terraform looks it up using `AWR_API_CLIENT_ID`
Required application-role value, normally `TalentMatch.Access`	`AWR_API_APP_ROLE_VALUE`; copy the role value, not its UUID or display name

The AWReason app role must allow **Applications** as a member type. Assign the role directly to the selected runtime identities only:

- Stack A managed identity principal object ID when `STACK_TARGET=stack-a` or `both`.
- Stack B managed identity principal object ID when `STACK_TARGET=stack-b` or `both`.

Do not grant this application role through a group. Entra does not emit an app role claim to a managed identity merely because its service principal is a member of a group that has the role.

Terraform creates one assignment per selected stack only when `AWR_AUTH_MODE=entra` and `AWR_API_CLIENT_ID` is populated. With `apikey` or `none`, skip this section: no AWReason Entra assignment is required. If the customer creates the assignments manually first, coordinate Terraform import before apply to avoid duplicate ownership.

Populate:

```
AWR_AUTH_MODE=entra
AWR_AAD_AUDIENCE=api://<awreason-api-client-id>/.default
AWR_API_CLIENT_ID=<awreason-api-application-client-id>
AWR_API_APP_ROLE_VALUE=TalentMatch.Access
```

If the actual AWReason Application ID URI is not `api://<client-id>`, use that exact URI with `/.default` appended in `AWR_AAD_AUDIENCE`.

13. Optional GitHub Actions Setup

GitHub Actions-related settings are required only if deployment will run through the repository's GitHub Actions workflow. They are not required for manual CLI deployment. Select the deployment method before creating a deployment application or federated credential.

Item	Manual CLI deployment	GitHub Actions deployment
GitHub deployment app and enterprise application	Not required; use the approved CLI operator/deployment identity	Required; reuse an approved identity or create one
GitHub OIDC federated identity credential	Not required; do not create	Required for each GitHub environment the identity deploys
GitHub OIDC secrets (`AZURE_CLIENT_ID`, `AZURE_TENANT_ID`, `AZURE_SUBSCRIPTION_ID`)	Not required in GitHub	Required in each deployed GitHub environment
Complete `DEPLOYMENT_ENV_FILE` environment secret	Not required in GitHub	Required; contains the approved deployment profile, including any Stack A packaging inputs
Target `.env` deployment profile	Required	Materialized from `DEPLOYMENT_ENV_FILE`; never copied from an example
Azure RBAC and Entra rights for deployment operations	Required for the actual CLI operator/deployment identity	Required for the GitHub deployment identity

For CLI-only deployment, skip sections 13.1, 13.2 and 13.5 and mark the GitHub-only acceptance items **Not applicable**. Apply the permission requirements in sections 13.3 and 13.4 to the actual operator/deployment identity instead; skipping GitHub does not remove those requirements.

GitHub OIDC credentials are separate from the optional Microsoft 365 connector's federated credentials. If `MICROSOFT365_SOURCE_MODE=graph`, the connector's federation remains required even when deployment uses the CLI.

13.1 Deployment App and Service Principal (GitHub Actions Only)

The GitHub workflow uses OpenID Connect. Do not create a client secret.

1. Reuse an approved deployment app and enterprise application, or create a single-tenant app registration named `talentmatch-<environment>-github-deploy`.
2. Ensure the corresponding enterprise application exists.
3. Record:
   - **Application (client) ID** -> GitHub environment secret `AZURE_CLIENT_ID`, not a SPA client ID variable or runtime identity value.
   - **Service principal Object ID** -> handoff only, for deployment Azure RBAC assignments; not the GitHub `AZURE_CLIENT_ID` secret.
   - **Directory (tenant) ID** -> GitHub environment secret `AZURE_TENANT_ID`; also `AZURE_TENANT_ID` in the deployment profile.

- **Subscription ID** from the target Azure subscription -> GitHub environment secret `AZURE_SUBSCRIPTION_ID`; also `AZURE_SUBSCRIPTION_ID` in the deployment profile.

This deployment app is separate from the TalentMatch protected API and selected SPA registration(s). Do not create it for CLI-only deployment.

13.2 Federated Identity Credentials (GitHub Actions Only)

The repository uses GitHub environments named:

- `development`
- `staging`
- `production`

Create one federated credential for each GitHub environment that the identity will deploy. Do not create credentials for unused environments:

Field	Value
Issuer	`https://token.actions.githubusercontent.com`
Audience	`api://AzureADTokenExchange`
Subject for development	`repo:wknums/talent-match-ui:environment:development`
Subject for staging	`repo:wknums/talent-match-ui:environment:staging`
Subject for production	`repo:wknums/talent-match-ui:environment:production`

Create separate deployment identities per environment if required by customer policy. This is preferred for production isolation.

13.3 Azure RBAC for the Deployment Identity

These permissions are required according to the resources being deployed, regardless of whether Terraform runs through the CLI or GitHub Actions. Grant them to the actual operator/deployment identity, not to an otherwise unused GitHub app.

For the full current Terraform deployment, grant:

Scope	Role	When required
Target deployment resource group	Contributor	Create and update App Service, plan, identities, and selected shared resources
Subscription	Contributor or approved custom role	Only if Terraform must create the resource group
Existing resource groups containing reused resources	Reader	Terraform data-source lookup
Existing VNet resource group	Reader	Reuse-only lookup
Existing VNet resource group	Network Contributor	Create/configure integration subnet, SQL private endpoint, or private DNS
Managed identity resources	Managed Identity Operator	Required when identity assignment is separated from broader Contributor access
Target App Services	Website Contributor	Sufficient for code-only ZIP deployment after infrastructure exists

Do not grant Owner unless the deployment must create Azure RBAC role assignments. The current Terraform creates key-scoped `Key Vault Crypto User` assignments for the trusted hosting managed identities when the Microsoft 365 source connector and its Key Vault wrapping key are enabled. The deployment identity therefore needs permission to create role assignments at that key scope, normally User Access Administrator or Role Based Access Control Administrator plus the resource-management permission required for the key. When `AZ_APP_INSIGHTS_ENABLED=TRUE`, Terraform also creates component-scoped `Monitoring Metrics Publisher` assignments for the selected runtime identities, including when the component is reused. This requires role-assignment permission at the Application Insights component scope. If neither feature needs Terraform-managed role assignments, broader role-assignment permission is not required.

13.4 Entra Permissions for the Deployment Identity

Entra rights depend on Terraform's writes, not on the use of GitHub. For an interactive CLI deployment, use an operator with the corresponding approved directory roles/object ownership; do not create a GitHub app or grant it Graph application permissions for that purpose. The application-permission guidance below applies to a noninteractive service principal, including the GitHub deployment identity when used.

When `ENTRA_REUSE=TRUE`, Terraform does not create the selected TalentMatch registrations, but the shared apply still expects to create:

- The bootstrap user's `admin` assignment on the protected API enterprise application.
- One managed identity assignment per selected stack on the AWReason enterprise application when `AWR_AUTH_MODE=entra`.
- Federated identity credentials on the reused Microsoft 365 source connector when `MICROSOFT365_SOURCE_MODE=graph`.

The noninteractive deployment identity therefore needs customer-approved Microsoft Graph application permissions for those writes, normally:

- `Application.Read.All`
- `AppRoleAssignment.ReadWrite.All`

Admin consent is required for application permissions.

If Terraform will create app registrations with `ENTRA_REUSE=FALSE`, it also requires application lifecycle permission, such as `Application.ReadWrite.OwnedBy` for objects owned by the deployment identity, or the broader `Application.ReadWrite.All` if approved by customer policy. The same ownership or application-write requirement applies when Terraform adds federated credentials to a reused connector application.

If Terraform will create groups, it additionally requires `Group.ReadWrite.All`. The manual group model in this document uses `ENTRA_CREATE_ROLE_GROUPS=FALSE`, so that permission is not required.

Where the customer will not grant Graph write permissions to the deployment identity, the relevant app-role assignments must be created manually and imported into the shared Terraform state before apply. Manual creation without state import is not a supported handoff because Terraform will attempt to own the same assignments.

13.5 GitHub Environment Secrets (GitHub Actions Only)

Only when using GitHub Actions, configure these secrets in each GitHub environment being deployed. They are not prerequisites for CLI-only deployment:

```
AZURE_CLIENT_ID=<github-deployment-app-client-id>
AZURE_TENANT_ID=<tenant-id>
```

```
AZURE_SUBSCRIPTION_ID=<subscription-id>
DEPLOYMENT_ENV_FILE=<complete-approved-deployment-profile-content>
```

`AZURE_CLIENT_ID` in GitHub means the GitHub deployment app client ID. `AZURE_CLIENT_ID` in each running App Service means that stack's user-assigned managed identity client ID. These values have the same setting name in different security contexts and must not be confused.

For CLI deployment, `AZURE_TENANT_ID` and `AZURE_SUBSCRIPTION_ID` still belong in the applicable `.env` profile, and Terraform still sets the runtime `AZURE_CLIENT_ID` for the selected managed identity. These Azure settings are not optional just because no matching GitHub secrets are configured.

Set `DEPLOYMENT_ENV_FILE` to the full multiline contents of the completed, approved `.env_local`, `.env_qa`, or `.env_prod` profile, not a filename or an unchanged example. Keep the secret in the corresponding GitHub environment. Its `ENVIRONMENT` must match the selected `dev`, `test`, or `prod` mapping, and its tenant/subscription must match the OIDC login secrets.

Target resolution, infrastructure deployment, and Stack A packaging all use this profile as their configuration source. CI writes it only to an untracked, permission-restricted file and refuses missing or empty secrets rather than falling back to examples. Never print or upload that file.

Include `ENTRA_API_SCOPE_ID`, all four `ENTRA_*_APP_ROLE_ID` values, and the other required profile inputs. Stack A packaging reads its selected SPA client ID and API scope settings from the same profile. Do not add Stack A inputs to a Stack B-only profile. Monitoring settings also belong in this profile; separate GitHub environment variables no longer override them.

Artifact deployment reads `RESOURCE_GROUP` from the profile and the App Service name from the selected subscription/environment Terraform workspace. Missing targets fail deployment; the workflow does not synthesize resource names.

14. Environment File Handoff

The customer identity and SQL teams must return the following values to the deployment team. The file is non-secret for the fields shown below, but it still contains controlled environment metadata and should use the customer's approved configuration-management process.

Copy each recorded value to the exact variable named in sections 5-13. Replace `<...>` placeholders with actual values; do not copy the placeholder brackets. The template below is the identity/SQL portion of the deployment profile, not a replacement for its other required deployment settings. Populate only the selected stack's inputs. Do not add `TF_VAR_*` variables: the deployment scripts generate them from these profile variables.

14.1 Recommended Manual-Preprovisioning Template

```
# Tenant and subscription
AZURE_TENANT_ID=<directory-tenant-id>
AZURE_SUBSCRIPTION_ID=<subscription-id>

# Application authentication
APP_AUTH_MODE=entra

# Customer pre-created Entra registrations
ENTRA_REUSE=TRUE
ENTRA_CREATE_ROLE_GROUPS=FALSE
```

```
ENTRA_API_APP_CLIENT_ID=<protected-api-application-client-id>
ENTRA_API_SERVICE_PRINCIPAL_OBJECT_ID=<protected-api-enterprise-app-object-id>
ENTRA_API_IDENTIFIER_URI=api://<protected-api-application-client-id>
ENTRA_API_SCOPE=access_as_user
ENTRA_API_SCOPE_ID=<existing-or-once-selected-delegated-scope-uuid>

# App-role IDs from the protected API registration
ENTRA_ADMIN_APP_ROLE_ID=<admin-app-role-id>
ENTRA_ORGANIZATION_ADMIN_APP_ROLE_ID=<organization-admin-app-role-id>
ENTRA_RECRUITER_APP_ROLE_ID=<recruiter-app-role-id>
ENTRA_BUSINESS_PANEL_APP_ROLE_ID=<business-panel-app-role-id>

# First administrator
ENTRA_BOOTSTRAP_ADMIN_OBJECT_ID=<enabled-member-user-object-id>
ENTRA_BOOTSTRAP_ORGANIZATION_NAME=<initial-organization-name>
ENTRA_BOOTSTRAP_DEPARTMENT_NAME=<initial-department-name>

# Customer pre-created runtime managed identities
AZ_IDENTITIES_REUSE=TRUE
AZ_IDENTITIES_RG=<managed-identity-resource-group>

# Reused Azure SQL
AZ_SQL_REUSE=TRUE
AZ_SQL_BOOTSTRAP_ENABLED=FALSE
SQL_SERVER_NAME=<logical-sql-server-name>
SQL_DATABASE_NAME=<application-database-name>
SQL_RG=<sql-resource-group>
SQL_AAD_ADMIN_LOGIN=<bootstrap-admin-user-principal-name>
SQL_AAD_ADMIN_OBJECT_ID=<same-value-as-ENTRA_BOOTSTRAP_ADMIN_OBJECT_ID>

# AWReason: use the approved environment mode; Entra inputs are separate below
AWR_AUTH_MODE=apikey
```

Append **exactly one** of these stack-specific blocks to the common settings above. Leave the other stack's fields absent or empty. Do not substitute the selected SPA's client ID into the unselected stack's field.

Stack A only:

```
STACK_TARGET=stack-a
ENTRA_STACK_A_REDIRECT_URIS=https://<stack-a-host>/
ENTRA_STACK_A_CLIENT_ID=<stack-a-spa-application-client-id>
AZ_IDENTITY_STACK_A_NAME=id-talentmatch-node-<environment>
```

Stack B only:

```
STACK_TARGET=stack-b
ENTRA_STACK_B_REDIRECT_URIS=https://<stack-b-host>/
ENTRA_STACK_B_CLIENT_ID=<stack-b-spa-application-client-id>
AZ_IDENTITY_STACK_B_NAME=id-talentmatch-blazor-<environment>
```

Both stacks:

```
STACK_TARGET=both
ENTRA_STACK_A_REDIRECT_URIS=https://<stack-a-host>/
ENTRA_STACK_B_REDIRECT_URIS=https://<stack-b-host>/
ENTRA_STACK_A_CLIENT_ID=<stack-a-spa-application-client-id>
ENTRA_STACK_B_CLIENT_ID=<stack-b-spa-application-client-id>
AZ_IDENTITY_STACK_A_NAME=id-talentmatch-node-<environment>
AZ_IDENTITY_STACK_B_NAME=id-talentmatch-blazor-<environment>
```

If AWReason uses Entra authentication, replace the common `AWR_AUTH_MODE` setting and add the following. Otherwise do not provision AWReason Entra assignments for the application. Supply any API key through the approved secret handoff, not this non-secret register.

```
AWR_AUTH_MODE=entra
AWR_AAD_AUDIENCE=api://<awreason-api-client-id>/.default
AWR_API_CLIENT_ID=<awreason-api-application-client-id>
AWR_API_APP_ROLE_VALUE=TalentMatch.Access
```

If the protected API uses a custom identifier URI, replace both `ENTRA_API_IDENTIFIER_URI` and the corresponding scope configuration with that exact URI.

If the identities remain Terraform-created, keep `AZ_IDENTITIES_REUSE=FALSE` and omit `AZ_IDENTITIES_RG` and the selected `AZ_IDENTITY_STACK_*_NAME` reuse coordinates. Terraform creates only the selected identities.

If the Microsoft 365 source connector is enabled, complete the additional registration, callback, publisher, federation, and Key Vault handoff in Delegated Microsoft 365 bulk-scoring connector registration. Do not infer connector activation from the product-registration values above.

14.2 Values Produced but Not Stored Directly in the `.env` File

Retain these values in the customer handoff register:

Output	Environment variable destination	Use
Protected API app registration object ID	None; handoff only. Not `ENTRA_API_SERVICE_PRINCIPAL_OBJECT_ID`	Application ownership and audit
Selected SPA app registration object IDs only	None; handoff only. Not `ENTRA_STACK_A_CLIENT_ID`/`ENTRA_STACK_B_CLIENT_ID`	Application ownership and audit
Selected SPA enterprise application object IDs only	None; handoff only	Consent and enterprise-app administration
Security group object IDs	None; application mappings, not deployment `.env` values	TalentMatch group-to-business-scope mappings
Managed identity resource IDs	No direct `.env` input; Terraform resolves them	Attach identities to App Services
Managed identity client IDs	Runtime `AZURE_CLIENT_ID` only, populated by Terraform	Per-stack managed identity credential selection
Managed identity principal object IDs	None; Terraform resolves them for assignments	AWReason app-role and Azure RBAC assignments
AWReason enterprise application object ID	None; Terraform looks it up using `AWR_API_CLIENT_ID`	AWReason app-role assignment target
Optional Microsoft 365 connector application/service principal object IDs	None; handoff only. Its client ID belongs in `MICROSOFT365_SOURCE_CLIENT_ID`	Connector ownership, consent and federation administration

Output	Environment variable destination	Use
GitHub deployment service principal object ID, only when GitHub Actions is used	None; handoff only. GitHub secret `AZURE_CLIENT_ID` takes its Application (client) ID instead	Azure RBAC assignments for that deployment identity; not required for CLI-only handoff

Terraform obtains and wires the managed identity client IDs to the App Services. The resulting runtime settings are:

```
AZURE_SQL_AUTH_MODE=entra
AZURE_SQL_SERVER_FQDN=<sql-server>.database.windows.net
AZURE_SQL_DATABASE_NAME=<application-database>
AZURE_CLIENT_ID=<that-stack-managed-identity-client-id>
```

Do not place managed identity principal object IDs in `AZURE_CLIENT_ID`. Do not overwrite the GitHub deployment app's `AZURE_CLIENT_ID` secret with a runtime managed identity client ID; these are separate identity contexts.

15. Acceptance Checklist

Entra Applications

- ☐ `STACK_TARGET` and authentication/reuse flags are agreed before creation.
- ☐ Product inventory is API plus one selected SPA (two registrations), or API plus both SPAs (three registrations); no unused stack objects are created.
- ☐ Protected API is single-tenant.
- ☐ API access-token version is 2.
- ☐ API identifier URI exactly matches `ENTRA_API_IDENTIFIER_URI`.
- ☐ Enabled delegated scope value is exactly `access_as_user`.
- ☐ Four enabled app roles have the exact required values.
- ☐ Protected API enterprise application object ID is recorded.
- ☐ Each selected frontend is a SPA/public-client registration.
- ☐ SPA redirect URIs exactly match the deployed origins.
- ☐ No SPA client secret or certificate exists.
- ☐ Each selected SPA has the delegated API permission.
- ☐ Admin consent is granted for each selected SPA.
- ☐ At least two durable owners exist on each application.

Groups and Application Roles

- ☐ Security groups are security-enabled and application-specific.
- ☐ Membership is direct, not nested.
- ☐ Each group has exactly one protected API app-role assignment.
- ☐ Group object IDs and business scopes are recorded.
- ☐ Bootstrap user is directly assigned the `admin` role.
- ☐ Group/user role assignments and Terraform ownership do not conflict.

Managed Identities and AWReason

- ☐ Only the selected user-assigned managed identities are created or reused.
- ☐ Resource IDs, client IDs, and principal object IDs are recorded separately.
- ☐ Each identity is attached only to its intended App Service.
- ☐ With `AWR_AUTH_MODE=entra`, each selected identity has a direct `TalentMatch.Access` assignment on the AWReason enterprise application.

- [ ] With AWReason API-key or no authentication, no AWReason Entra assignment is provisioned for the application.
- [ ] Neither identity has a client secret.

Azure SQL

- [ ] SQL Entra administrator is the enabled Member bootstrap user.
- [ ] SQL Entra administrator object ID equals `ENTRA_BOOTSTRAP_ADMIN_OBJECT_ID`.
- [ ] Bootstrap host resolves and reaches the private SQL endpoint.
- [ ] Each managed identity selected by `STACK_TARGET` exists as a contained user in the application database.
- [ ] Each selected identity is a member of `db_datareader`, `db_datawriter`, and `db_ddladmin`.
- [ ] Neither runtime identity is `db_owner`.
- [ ] Each selected application can connect without a SQL password.

Deployment Access (Required for Either Deployment Method)

- [ ] CLI or GitHub Actions is explicitly selected as the deployment method.
- [ ] Permissions are granted to the actual operator/deployment identity.
- [ ] Azure RBAC is scoped to required resource groups/resources.
- [ ] Network Contributor is granted only where network changes are required.
- [ ] Noninteractive deployments have admin consent for required Graph application permissions; interactive CLI operators have the corresponding approved directory rights.
- [ ] Temporary directory and Azure privileges are removed after handoff.

GitHub Actions Only (Otherwise Mark Not Applicable)

- [ ] For CLI-only deployment, no GitHub-only app, federated credential, secret or variable is created just to satisfy this guide.
- [ ] If GitHub Actions is used, its deployment app is separate from runtime applications and is created or reused only for environments being deployed.
- [ ] If GitHub Actions is used, its OIDC federated credential issuer, audience, and subjects are exact, with no deployment client secret.
- [ ] If GitHub Actions is used, deployed GitHub environments contain the required deployment app client ID, tenant ID, and subscription ID secrets.
- [ ] If GitHub Actions packages Stack A with Entra authentication, the required public packaging variables are configured; they are not required for Stack B-only deployment.

Environment Handoff

- [ ] `ENTRA_REUSE=TRUE` is set for manually created registrations.
- [ ] Reused-registration IDs include the shared API client ID, API service principal object ID and selected SPA client ID(s): three IDs for a single stack, four for both.
- [ ] All four app-role IDs match the protected API manifest.
- [ ] Redirect URI lists are populated only for the selected SPA(s).
- [ ] Managed identity reuse coordinates match the created identities.
- [ ] Reused SQL coordinates are correct.
- [ ] AWReason audience, client ID, and role value are correct if Entra mode is enabled.
- [ ] The optional Microsoft 365 connector is provisioned only for `graph` mode, with federation and key access scoped to selected hosting identities.
- [ ] The reviewed plan creates no unselected-stack components and contains no unexpected deletion of existing managed objects.
- [ ] No passwords, tokens, refresh tokens, or SPA secrets are present.

16. Security Notes

- Use managed identities and OIDC instead of client secrets.
- Treat redirect URIs as exact security boundaries.
- Grant application permissions and admin consent only after review.
- Prefer object ownership or resource-scoped roles over standing tenant-wide administrator roles.
- Use PIM and time-bound assignments for elevated implementation tasks.
- Do not grant application users Azure RBAC or direct SQL access.
- Do not infer authorization from Entra group names. Use immutable group object IDs and explicit application mappings.
- Do not assume an Entra role claim alone grants application access. TalentMatch also requires active organization and department authorization in the shared database.
- Store no client secrets for the SPAs, managed identities, or GitHub OIDC deployment.

17. Repository References

- Entra authorization operations
- Authentication behavior
- Infrastructure deployment runbook
- Azure SQL managed identity bootstrap helper
- Entra Terraform module
- Managed identity Terraform module
- Azure SQL Terraform module
- GitHub deployment workflow
- Delegated Microsoft 365 bulk-scoring connector registration

18. Microsoft References

- Add app roles to an application and receive them in the token
- Manage users and groups assignment to an application
- Single-page application registration and configuration
- Deploy to Azure App Service by using GitHub Actions
- Connect GitHub and Azure with OpenID Connect
- Microsoft Entra service principals with Azure SQL
- Managed identities for Azure resources
- Azure built-in roles